ECEN 248 - Lab Report

Lab #6

Introduction to Behavioral Simulation and Logic Synthesis

Section: 524

Ramya Kannan

UIN: 635002011

10/30/2025

TA: Reed Clardy

Objectives:

The objective of this is to introduce a higher level of abstraction available in Verilog HDL, known as behavior modeling. In this lab, we will recreate multiplexers, then use behavioral Verilog to describe the binary encoders and decoders. Along with continuing to introduce topics in Verilog, we will be introduced to logic synthesis which is a process of translating HDL code into implementable digital logic. The goal of this lab is to gain skills in writing Verilog code, and utilizing the testbenches for circuit verification, and understand the concepts of behavioral modeling and logic synthesis.

Design:

This lab implemented behavioral modeling, logic synthesis, and FPGA implementation using Verilog HDL in the Xilinx Vivado Design Suite. The design process began with simulating the 1- bit 2:1 mux named two_one_mux_behavioral with output Y declared as a type reg and inputs A, B, and S as wires. We use type reg because they are required for variables assigned within always blocks. An always block triggered by changes in A, B, or S changes used an if-else statement: if S equaled 1'b0, Y was assigned A, otherwise B. This module was verified with two_one_mux_tb.v

Next, we extended the 2:1 mux to a 4-bit 2:1 Mux and called the module four_bit_mux. Y is a 4-bit reg type and A, B as 4-bit wires. The 1-bit S (select) and if-else structure remained unchanged, handling all four bits simultaneously. This module was tested with the four_bit_mux_tb.v. Then, we extended the 4-bit 2:1 mux to a 4-bit 4:1 mux and called the module mux_4bit_4to1. The mux has four 4-bit inputs A, B, C, D and a 2-bit S (Select). The always@(*) block used a case statement: 2'b00 selected A, 2'b01 selected B, 2'b10 selected C, 2'b11 selected D. We verified this module with the mux_4bit_4to1_tb.v.

In the next part of the lab we created the module two_four_decoder module which had a 4-bit output Y as type reg, and a 2-bit input W, and enabled En. The always block verified if En = 1'b1, then decoded W with this case statement: 2'b00 producing 4'b0001, 2'b01 producing 4'b0010, 2'b10 producing 4'b0100, and 2'b11 producing 4'b1000. Otherwise, Y was 4'b0000. This module was tested with two_four_decoder_tb.v Then we created the four_two_encoder module which consisted of a 4-bit input W, 2-bit output Y as type reg, and zero output. The zero signal used assign zero = (W ==4'b0000). The always block case statement is : 4'b0001 to 2'b00, 4'b0010 to 2'b01, 4'b0100 to 2'b10, and 4'b1000 to 2'b11, with default 2'bXX. This module was tested and verified with four_two_encoder_tb.v

Next, we created the priority_encoder module which used casex for don't-care handling. 4'b0001 produced 2'b00, 4'b001X produced 2'b01, 4'b01XX produced 2'b10, and 4'b1XXX produced 2'b11, giving w[3] highest priority. We verified and tested the priority_encoder module with the priority_encoder_tb.v. The last part of the experiment consisted of a physical implementation through the usage of an FPGA. All the modules were mapped to a portion of the ZYBO Z7-10 using XDC files. For the two_four_decoder, the outputs were set to LED3-LED0. For the priority encoder, the XDC files mapped W[3:0] to push buttons, allowing us to physically test and verify it.

Results:

All behavioral Verilog modules were successfully simulated in Vivado and verified using their corresponding testbenches. The 1-bit 2:1 multiplexer correctly selected between inputs A and B based on select signal S, passing all eight test combinations. The 4-bit 2:1 MUX and 4-bit 4:1 MUX simulations confirmed proper operation across all test cases, with waveforms showing correct output selection.

The 2:4 binary decoder simulation validated the enable and decoding logic. When En was low, all outputs remained 4'b0000. When En was high, the decoder produced correct one-hot encoded outputs for each 2-bit input combination. For example, when W = 2'b10, the output Y = 4'b0100, lighting only the third bit. The 4:2 binary encoder correctly converted one-hot inputs to binary outputs and properly asserted the zero signal when no inputs were active. However, when multiple input bits were high simultaneously, the encoder produced don't-care outputs (2'bXX).

The priority encoder simulation demonstrated correct priority-based encoding. For single active inputs, it behaved identically to the binary encoder. When multiple inputs were active, it correctly encoded the highest-priority bit using the casex statement implementation. For example, when W = 4'b1010, the output was Y = 2'b11, corresponding to the highest-priority bit w[3]

FPGA hardware testing on the ZYBO Z7-10 board confirmed all simulation results. The 2:4 decoder was tested using switches for inputs and LEDs for outputs, producing correct one-hot encoded results. The 4:2 binary encoder used push-buttons and displayed correct binary codes for single button presses but showed unpredictable outputs when multiple buttons were pressed. The priority encoder successfully handled multiple simultaneous button presses by encoding the highest-priority active input.

Conclusion:

In this lab, we successfully introduced behavioral Verilog modeling and FPGA implementation for higher level abstraction in digital circuit design. All the modules were synthesized and successfully implemented on the FPGA board. Behavioral modeling using always blocks, case statements, and casex constructs proved more efficient than gate-level descriptions. Through testbench simulations and verification on the ZYBO Z7-10 board, we gained skills in writing Verilog code, performing logic synthesis, and translating HDL descriptions into functional digital circuits. Overall, this lab provided essential experience in digital design from behavioral modeling to FPGA implementation.

Post-Lab Deliverables:

1. **Code for all the modules**

Figure 1. 4-bit 2:1 Mux (Behavioral)

```
`timescale 1ns/ 1ps
`default_nettype none

module two_one_mux(Y, A, B, S);

    //declare out and input
    output reg [3:0 Y; //reg since it will be modifed in an always block
    input wire [3:0] A, B; //A and B are 4 bit wide wires
    input wire S; //select is still bit wde

    always@(A or B or S) //always block which triggers whenever A, B,or S changes
        begin
            if (S == 1'b0)
                Y = A; //drive Y with a
            else
                Y = B; //instead drive y with B
        end
endmodule //designate end of module
```

Figure 2. 4-bit 4:1 Mux

```
`timescale 1ns/ 1ps
`default_nettype none

module mux_4bit_4to1(Y, A, B, C, D, S);
    //declare output and intput ports
    output reg [3:0] Y;
    input wire [3:0] A, B, C, D; //4-bit wide input wires
    input wire [1:0] S;

    //select is a 2-bit wire

    always@(*) //new Verilong trick, means trigger when anything hanges
        case(S) //selection based on casex
            2'b00 : Y = A; //when S ==2 ' b00
            2'b01 : Y = B;
            2'b10 : Y = C;
            2'b11 : Y = D;
        endcase //desogmates the end of case statement
endmodule //de
```

Figure 3. 2:4 Binary Decoder

```
`timescale 1ns/ 1ps

module two_four_decoder(
    input wire [1:0] W,
    input wire En,
    output reg [3:0] Y);

    //behavioral portion
    always@ (En or W)
        begin
            if(En == 1'b1)
                case(W) //select based on W
                    2'b00 : Y=4'b0001; //2'b signifies a 2bit binary value
                    2'b01 : Y=4'b0010; //w is lit up
                    2'b10 : Y=4'b0100; //w is lit up
                    2'b11 : Y=4'b1000;
                endcase
            else
                Y = 4'b0000;
        end
endmodule
```

Figure 4. 4:2 Binary Encoder

```
`timescale 1ns/ 1ps
`default_nettype none

module four_two_encoder(
    input wire [3:0] W,
    output wire zero,
    output reg [1:0] Y
    );

    //assing case for no W
    assign zero = (W == 4'b0000);

    //behavioral logic for 4:2 enoder
    always@(W)
        begin
            case(W) //triggers when W changes
                4'b0001: Y = 2'b00;
                4'b0010: Y = 2'b01;
                4'b0100: Y = 2'b10;
                4'b1000: Y = 2'b11;
                default: Y = 2'bXX;
            endcase
        end
endmodule
```

Figure 5. Priority Encoder

```
`timescale 1ns/ 1ps
`default_nettype none

module priority_encoder(
    input wire [3:0] W,
    output wire zero,
    output reg [1:0] Y
    );

    //assing case for no W
    assign zero = (W == 4'b0000);

    //behavioral logic for 4:2 enoder
    always@(W)
        begin
            casex(W) //triggers when W changes
                4'b0001: Y = 2'b00;
                4'b001X: Y = 2'b01;
                4'b01XX: Y = 2'b10;
                4'b1XXX: Y = 2'b11;
                default: Y = 2'bXX;
            endcase
        end
endmodule
```

2. Waveforms

Figure 6. Waveform for 4-bit 2:1 Mux (Behavioral)

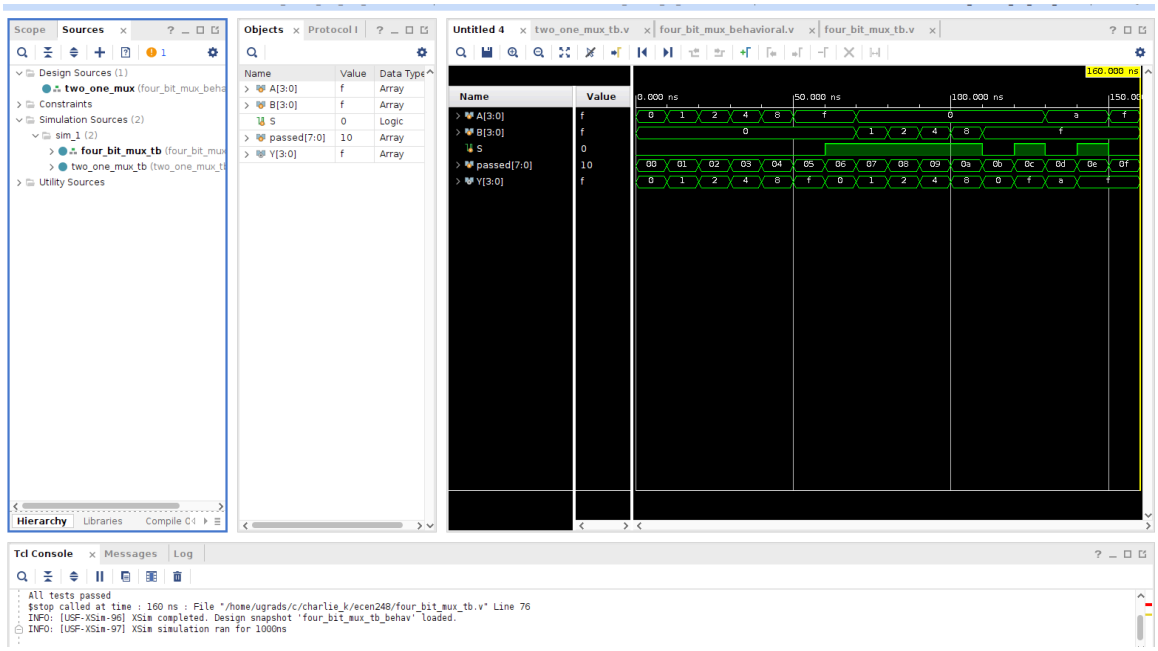


Figure 7. Waveform bit 4-bit 4:1 Mux

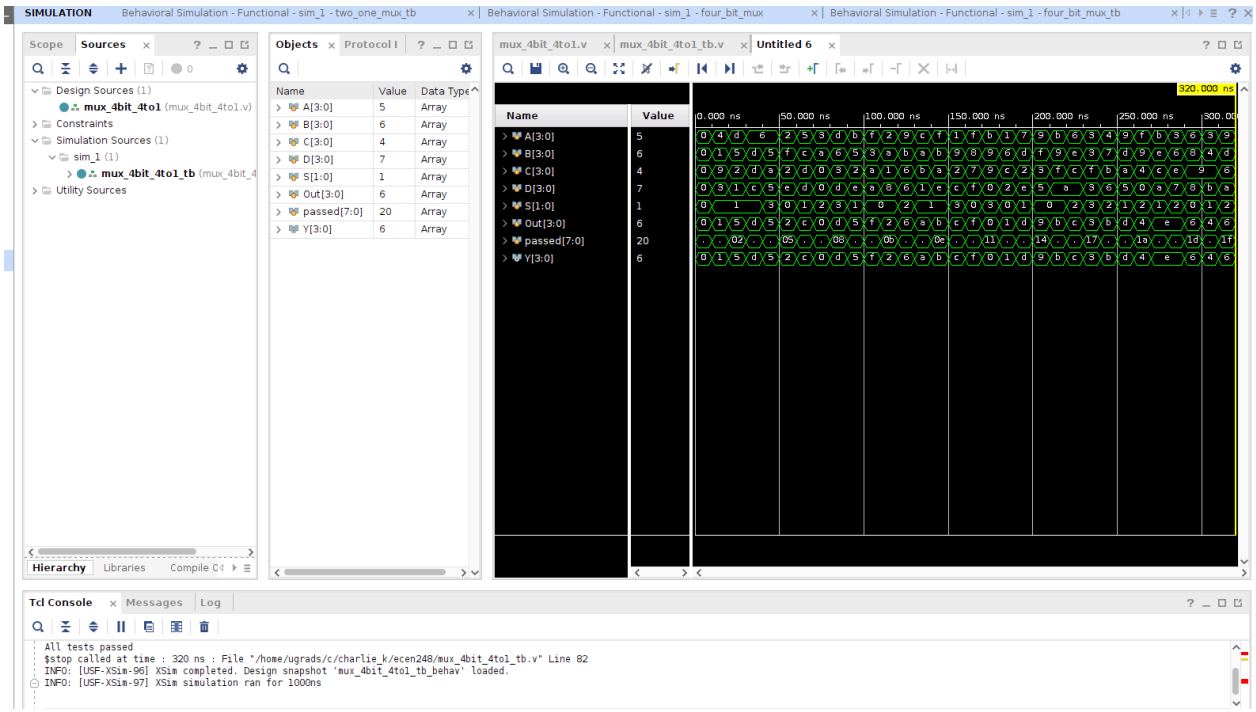


Figure 8. 2:4 Decoder

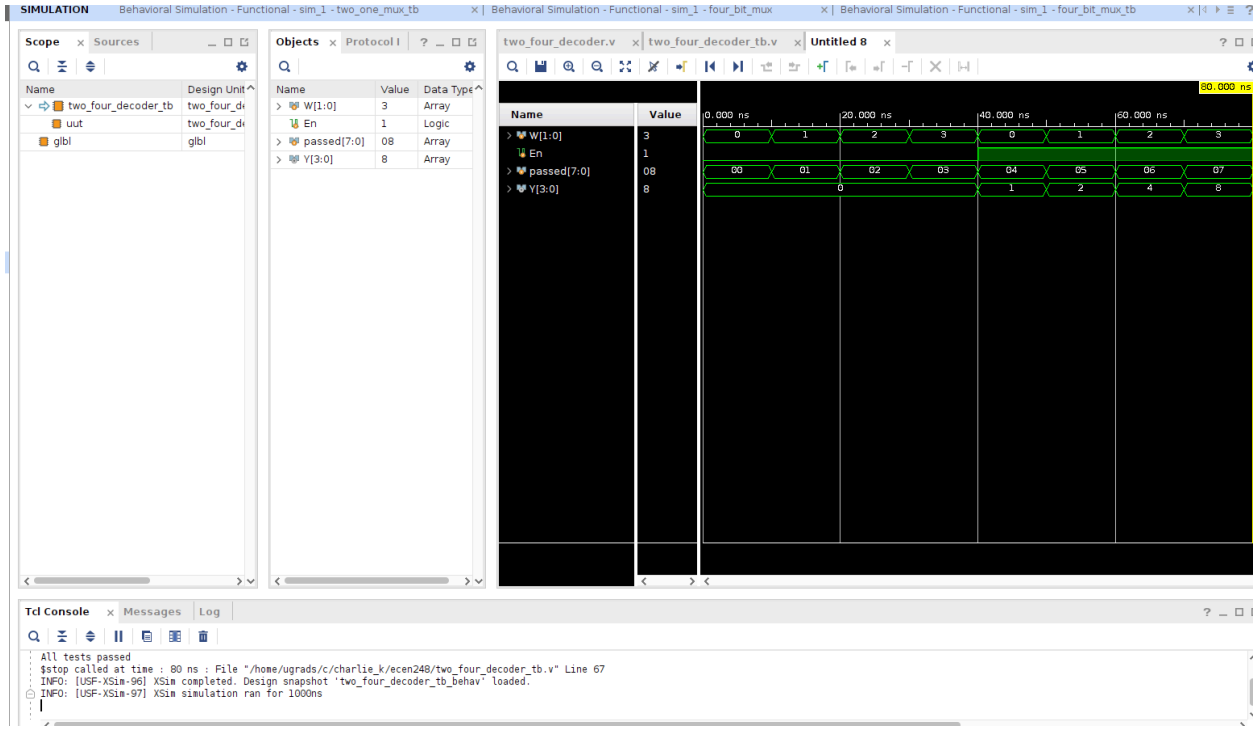


Figure 9. 4:2 Encoder

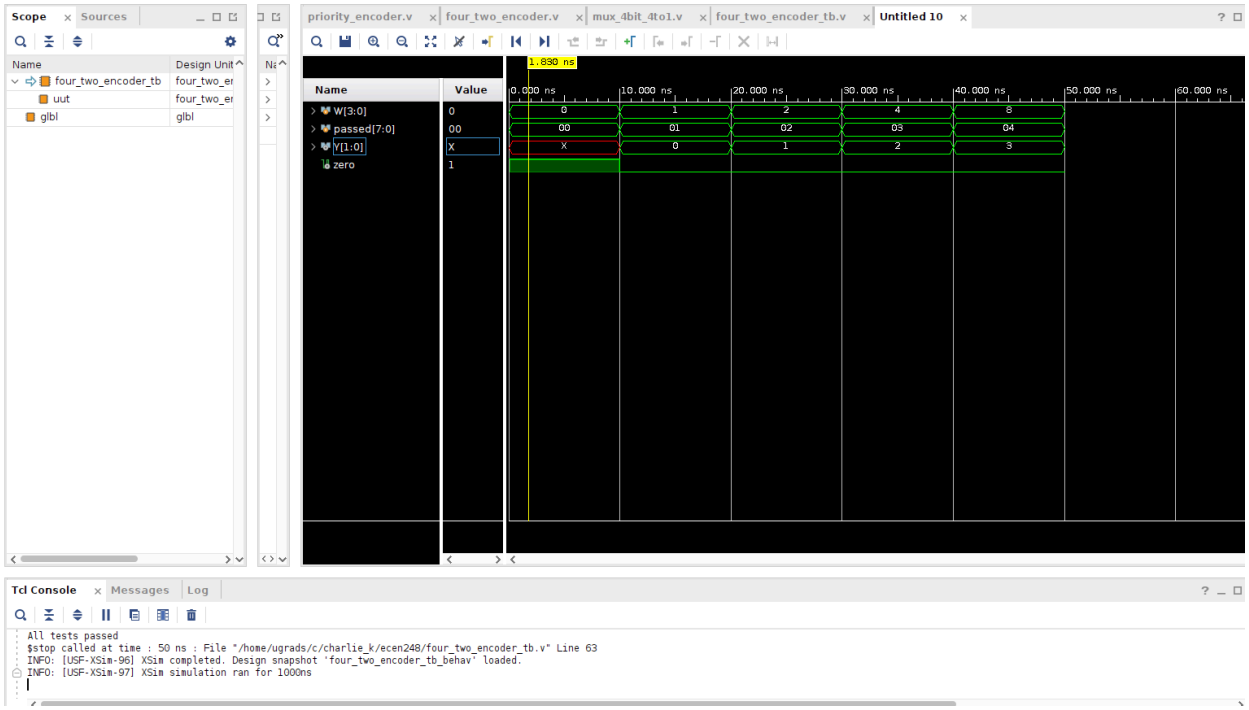
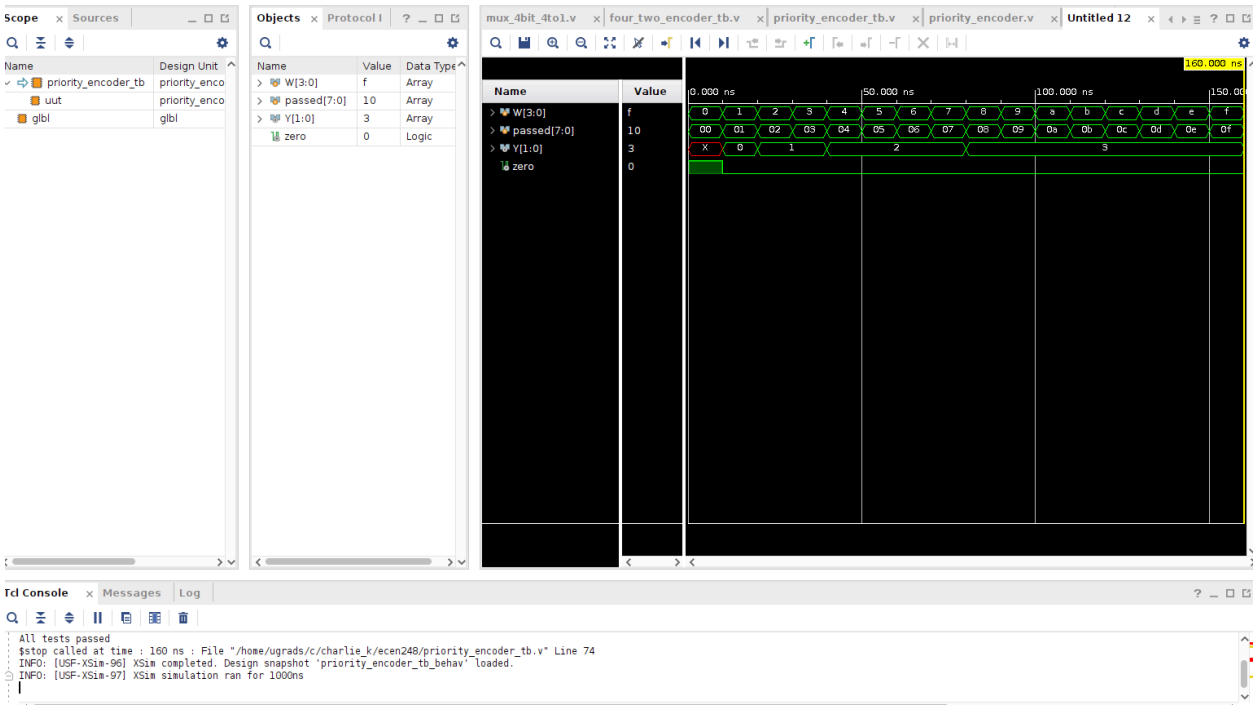


Figure 10. Priority Encoder



3. **Provide a comparison between behavioral Verilog used in this week's lab and the structural and dataflow Verilog used in last week's lab. What might be the advantages and disadvantages of each?**

Behavioral Verilog uses high-level programming constructs like if-else statements, case statements, and always blocks to describe circuit functionality. In this lab, the multiplexers and encoders used behavioral modeling with case statements to implement their logic. The advantage is that behavioral code focuses on what the circuit accomplishes rather than gate-level implementation, making it more readable and easier to scale. The disadvantage is that it abstracts away from actual hardware, and the reg type can be confusing since it doesn't always represent physical registers.

Structural Verilog instantiates gates and modules with explicit wire connections. In the previous lab, the 1-bit mux used gate primitives and the 4-bit mux instantiated four 1-bit instances. This provides clear visibility of circuit hierarchy and component interconnections. Dataflow Verilog uses assign statements with Boolean expressions. In Lab 5, the full adder used dataflow with equations like S = A ^ B ^ Cin. Both structural and dataflow offer precise control over gate-level implementation and show exactly what hardware is created. However, they become verbose and difficult to scale for wider designs compared to behavioral modeling.

4. **Compare the process of using a breadboard to implementing a digital circuit on an FPGA. State some advantages and disadvantages of each. Which process do you prefer?**

The advantages of using breadboarding are providing hands-on learning with the physical components (jumper wires, wires, LED's, etc). The visual feedback is instant with the LED outputs and allows us to improve our troubleshooting skills. However, breadboarding is extremely tedious and prone to error. Wires can break all the time, and sometimes we can confuse the wires. Testing multiple cases can be impractical. I agree with this; it is very tedious and impractical for FPGA implementation.

FPGA implementation is a lot more efficient than breadboarding. We still have the part where we need to write Verilog code and demonstrate the simulation to verify the functionality before we switch to the FPGA. The XDC files enable physical testing without manual wiring, and if we need to modify them, we can simply adjust the code, resynthesize, and reprogram it. This allows rapid testing for more complex design. I feel that you still get some hands-on learning through FPGA. I prefer FPGA implementation over breadboarding, especially dealing with the designs/circuits we created in this lab.

<u>Student Feedback</u>

1. **What did you like most about the lab assignment and why? What did you like least about it and why?**

   I really enjoyed the last part of the lab implementing the FPGA. There is nothing I didn't like

2. **Were there any sections of the lab manual that were unclear? If so, what was unclear? Do you have any suggestions for improving the clarity?**

   No, everything was clear.

3. **What suggestions do you have to improve the overall lab assignment?**

   No, I have no suggestions.